



LeetCode 1512 — Number of Good Pairs

1. Problem Title & Link

Title: LeetCode 1512: Number of Good Pairs

Link: <https://leetcode.com/problems/number-of-good-pairs/>

2. Problem Statement (Short Summary)

Given an array `nums`, a pair (i, j) is called a good pair if:

`nums[i] == nums[j]`

and

$i < j$

Return the total number of good pairs.

3. Examples (Input → Output)

Example 1

Input:

`nums = [1,2,3,1,1,3]`

Output:

4

Explanation:

Good pairs:

$(0,3) \rightarrow 1 = 1$

$(0,4) \rightarrow 1 = 1$

$(3,4) \rightarrow 1 = 1$

$(2,5) \rightarrow 3 = 3$

Total:

4

Example 2

Input:

`nums = [1,1,1,1]`

Output:

6

Explanation:

$(0,1)$



(0,2)

(0,3)

(1,2)

(1,3)

(2,3)

Total:

6

Example 3

Input:

nums = [1,2,3]

Output:

0

No equal pairs.

4. Constraints

$1 \leq \text{nums.length} \leq 100$

$1 \leq \text{nums}[i] \leq 100$

Important:

Array size is small

But we should still learn the optimal approach.

5. Core Concept (Pattern / Topic)

HashMap / Frequency Counting

This is one of the easiest frequency-counting problems.

6. Thought Process (Step-by-Step Explanation)

Brute Force Approach

Check every pair.

for i



for j

if nums[i] == nums[j]

Example:

[1,2,3,1,1,3]

Compare:

1 with all

2 with all

3 with all

Time Complexity:

$O(n^2)$

Works but inefficient.

Optimized Thinking

Suppose we process:

[1,2,3,1,1,3]

When we see first:

1

Frequency:

1 → 1

No pair yet.

See second:

1

Frequency already:

1

So:

New Good Pairs = 1

See third:

1

Frequency already:

2

So:



New Good Pairs = 2

Total:

$$1 + 2 = 3$$

for value 1.

This gives an $O(n)$ solution.

7. Visual / Intuition Diagram (ASCII)

```
nums
[1,2,3,1,1,3]
Processing:
1 → freq=0 → pairs +=0
2 → freq=0 → pairs +=0
3 → freq=0 → pairs +=0
1 → freq=1 → pairs +=1
1 → freq=2 → pairs +=2
3 → freq=1 → pairs +=1
Total:
4
```

8. Pseudocode

```
pairs = 0
map = {}
for num in nums
    pairs += map[num]
    map[num]++
return pairs
```

9. Code Implementation

Python

```
class Solution:
    def numIdenticalPairs(self, nums):
```



```
freq = {}

pairs = 0

for num in nums:

    pairs += freq.get(num, 0)

    freq[num] = freq.get(num, 0) + 1

return pairs
```

Java

```
import java.util.*;

class Solution {

    public int numIdenticalPairs(int[] nums) {

        HashMap<Integer,Integer> map = new HashMap<>();

        int pairs = 0;

        for(int num : nums) {

            pairs += map.containsKey(num) ? map.get(num) : 0;

            map.put(num, map.containsKey(num) ? map.get(num) + 1 : 1);

        }

        return pairs;

    }

}
```

10. Time & Space Complexity

Time Complexity

$O(n)$

Single traversal.

Space Complexity

$O(n)$

HashMap stores frequencies.

11. Common Mistakes / Edge Cases

**Mistake 1**

Incrementing frequency before counting pairs.

Wrong order:

```
freq++;
```

```
pairs += freq;
```

Produces incorrect answer.

Mistake 2

Using nested loops unnecessarily.

Mistake 3

Counting the same pair twice.

12. Detailed Dry Run

Input:

nums = [1,2,3,1,1,3]

Num	Existing Freq	New Pairs	Total Pairs
1	0	0	0
2	0	0	0
3	0	0	0
1	1	1	1
1	2	2	3
3	1	1	4

Final Answer:

4

13. Common Use Cases (Real-Life / Interview)**Duplicate Counting**

Examples:

Student ID duplicates

Transaction duplicates

Repeated log entries

Repeated usernames



14. Common Traps (Important!)

Trap 1

Using combinations formula without understanding frequency.

Trap 2

Updating frequency before counting.

Trap 3

Trying to store every pair explicitly.

Not required.

15. Builds To (Related LeetCode Problems)

LC 217 Contains Duplicate

↓

LC 1512 Number of Good Pairs

↓

LC 1207 Unique Number of Occurrences

↓

LC 49 Group Anagrams

↓

LC 347 Top K Frequent Elements

HashMap progression.

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Nested Loops	$O(n^2)$	$O(1)$	Easy
Frequency Array	$O(n)$	$O(100)$	Best for constraints
HashMap	$O(n)$	$O(n)$	General Solution

Approach 1: Nested Loops

for i in range(n):

 for j in range(i+1,n):

Simple but slow.

Approach 2: Frequency Counting



Count occurrences while traversing.

Optimal.

17. Why This Solution Works (Short Intuition)

When a number appears again, it forms a new pair with every previous occurrence of the same number.

If frequency is:

k

then the next occurrence creates:

k new pairs

Adding these counts gives the total answer.

18. Variations / Follow-Up Questions

Follow-Up 1

Return all pairs instead of count.

Follow-Up 2

Count pairs with difference = k .

Leads to hashing problems.

Follow-Up 3

Count pairs whose sum equals target.

Leads to Two Sum.

Follow-Up 4

Find most frequent element.

Leads to frequency-based problems.